

Objectives

After completing this lab, I will be able to:

- Understand Amazon VPC configuration
- Validate Subnet and Internet Gateway infrastructure
- Confirm a Route Table has a Public Route to the Internet
- Create Security Group rules
- Launch an Amazon Elastic Compute Cloud (Amazon EC2) instance
- Configure an EC2 instance to host a web application using a user data script

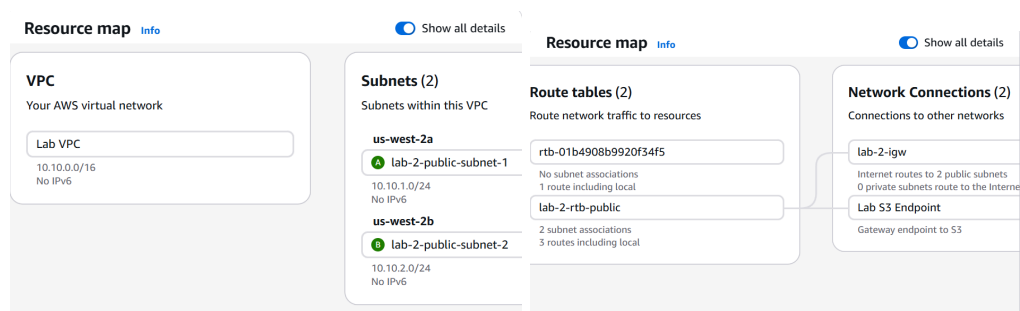
Scenario

In this scenario, I'll review the underlying network architecture that has been created for me, and is needed to run a web application on an Amazon EC2 instance. I'll then launch the EC2 instance used to host my web application and conduct a simple test to verify I can access it in a web browser.

Task 1: Understand a Virtual Private Cloud's Configuration

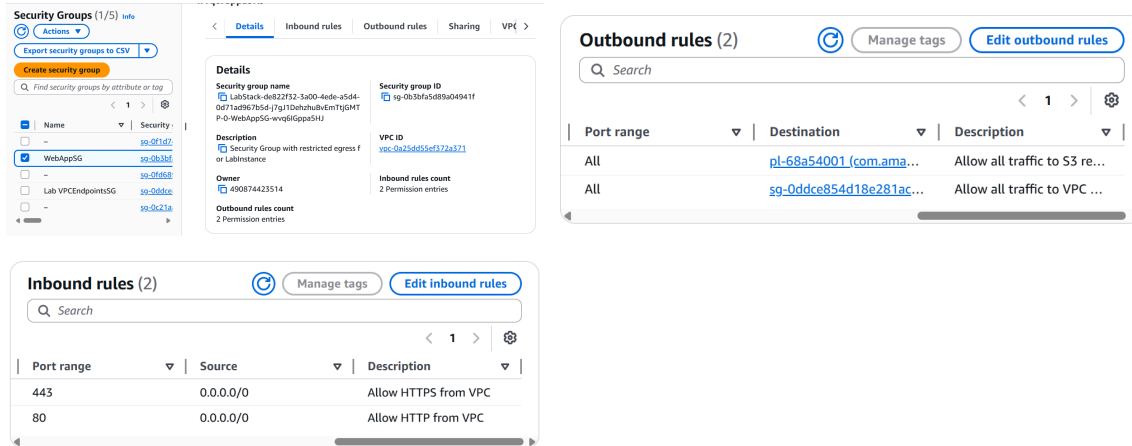
In this task, I'll review an Amazon VPC that has been created for me, and all the infrastructure attached to and within it. I'll look at the configuration of its internet gateway, route table, and subnets.

After creating a VPC, I'll add a *subnet* or multiple subnets. Each subnet resides entirely within one Availability Zone and cannot span multiple Availability Zones. A subnet is a range of IP addresses in my VPC. I can launch AWS resources into a specified subnet. I used a *public subnet* for resources that must be connected to the internet, and a *private subnet* for resources that won't be connected to the Internet. I will not use *private subnets* in this lab.



Task 2: Create a VPC Security Group

In this task, I'll review the VPC security group pre-created to launch my web application instance. A security group controls the traffic that is allowed to reach and leave the resources that it is associated with. For example, after I associate a security group with an EC2 instance, it controls the inbound and outbound traffic for the instance, similar to a firewall.



Task 3: Launch Your Amazon EC2 Instance

In this task, I created an EC2 instance and provided a bootstrap script to install and configure the requirements for my web application. I also enable SSH (Secure Shell) access to the instance.

```
#!/bin/bash -xe

# Installs Node.js
dnf install nodejs24 nodejs24-npm -y

# Downloads an NPM cache from S3 to aid package installation
aws s3 cp s3://npm-us-west-2-4045868732/npm-cache.tar.gz \
/var/cache/npm-cache.tar.gz

# Extracts the cache to a directory
mkdir -p /root/.npm

tar xzf /var/cache/npm-cache.tar.gz -C /root/.npm/

# Downloads the web app code as a zip file
mkdir -p /var/app/

aws s3 cp s3://npm-us-west-2-4045868732/app.zip \
/var/app/app.zip

# Extracts the web app zip to a directory
unzip /var/app/app.zip -d /var/app/
```

```
# Runs an offline npm install for needed packages
```

```
cd /var/app
```

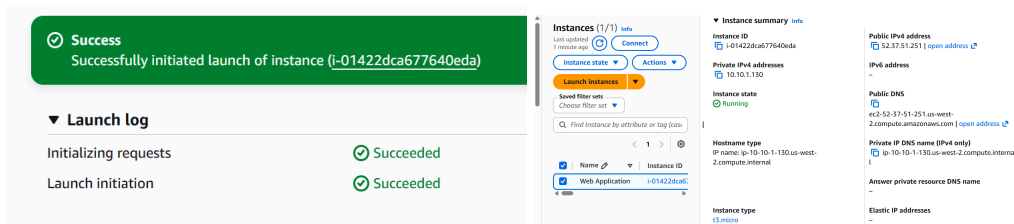
```
npm install --offline
```

```
# Starts the Node.js web app
```

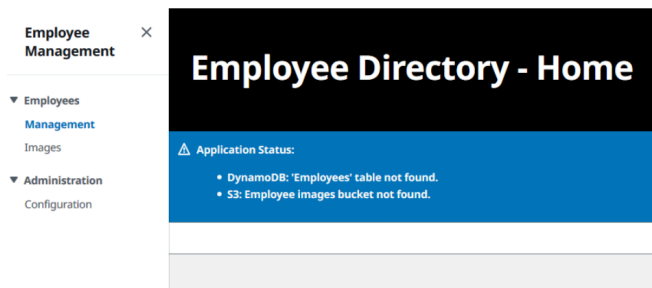
```
npm start
```

This script performs the following tasks:

- Installs Node.js
- Creates symbolic links for Node.js and Node Package Manager (NPM)
- Downloads an NPM cache from S3 to aid package installation
- Extracts the cache to a directory
- Downloads the web app code as a zip file
- Extracts the web app zip to a directory
- Runs an offline npm install for needed packages
- Starts the Node.js web app



My web application:



Lab Complete

I successfully did the following:

- Understood Amazon VPC configuration
- Validated Subnet and Internet Gateway infrastructure
- Confirmed a Route Table has a Public Route to the Internet
- Created Security Group rules

- Launched an Amazon Elastic Compute Cloud (Amazon EC2) instance
- Configured an EC2 instance to host a web application using a user data script

The following diagram represents the completed lab environment:

